

GCP dispatch bug

Problem

A Ray task the GCS assigns to the GCP worker node never reaches that worker's raylet, under this cluster's tailnet-relay topology (`cluster/local.yaml` , `tailnet: manage_all: true`).

Confirmed with a patient (6-minute) wait, not a premature kill: the task sits pending the whole time. The GCP worker's own raylet state-dump, taken during that window, shows:

- `num PYTHON pending start requests: 0`
- `num PYTHON pending registration requests: 0`
- every `process_failed_*` counter at 0

The raylet's WorkerPool is never even asked to start a worker. Nothing fails partway; nothing is attempted at all. The worker node itself shows `Alive` with correct resources the whole time — its own outbound heartbeat/registration to the GCS works fine. Only inbound (GCS -> worker) traffic is affected.

What's already ruled out (checked today, before spending more GCP time)

1. **SSH tunnel subprocess dying after `chia up` exits.** Found a real upstream fix for exactly this (`chia commit c4b6507` , branch `fix/persistent-cluster-ssh-tunnels` , unmerged). Turned out not to apply: that code path (`chia/cluster/tunnel.py` , `TunnelManager`) only runs when `config.is_tunneled(ip)` is true, which requires an explicit `auth.overrides.<ip>.tunnel` entry. Our config has none — every non-head worker gets `override.tailnet = True` instead (`chia/cluster/config.py` ~line 872), routing through `chia/cluster/tailnet.py` , a completely different, subprocess-free mechanism. Confirmed by reading both files directly.
2. **Node manager port mismatch.** Checked whether `worker_start_ray_commands` in `cluster/local.yaml` (`ray start --address=$RAY_HEAD_IP:6379 --dashboard-agent-listen-port=0`) tells Ray to bind the SAME port the tailnet relay's routing table expects. It does -- `chia` auto-injects `--node-manager-port` , `--node-ip-address` , `--object-manager-port` , `--min/max-worker-port` into any matching `ray start --address=...` line (`chia/cluster/node_setup.py` ~line 544-560), regardless of what the user's own YAML contains. Verified this applies to our exact command.

Both looked like strong candidates from the commit/config history alone. Neither holds up under actually reading the code. Recorded here so nobody re-chases either one.

How the relay actually routes (confirmed by reading `chia/cluster/tailnet.py`)

- Every tailnet node runs a local relay: a single HTTP CONNECT proxy listener on `127.0.0.1:<connect_proxy_port>` . Ray's gRPC (via `grpc_proxy=http://127.0.0.1:<port>` , `RAY_grpc_enable_http_proxy=1` , both confirmed present on real nodes) routes every outbound call through it.
- The relay maps the destination (an "advertise IP", a loopback address like `127.0.0.x` used as a routing key) to the owning machine's real tailnet IP, then dials out through that machine's own local `tailscaled` SOCKS5 proxy.
- Inbound tailnet traffic needs no relay on the receiving end: userspace `tailscaled` is supposed to deliver it straight to `127.0.0.1:<port>` , where Ray's wildcard-bound services receive it directly.
- The relay logs every CONNECT attempt and every dial failure to `/tmp/chia_tailnet_relay_$USER.log` on each machine.

So for a GCS-to-worker lease assignment: head's relay gets a CONNECT for the worker's advertise IP -> dials the worker's real tailnet IP via the head's own SOCKS5 proxy -> arrives at the worker's `tailscaled` -> should be delivered locally to the raylet's port. Two hops, either of which could be where this actually breaks -- and nothing checked so far actually looked at either relay's own log during a real dispatch attempt.

What's running right now

`scripts/patient_gcp_dispatch_test.py` , extended today to capture both relays' log output bracketing the dispatch window (previously it only checked the raylet's own internal counters after the fact). This directly answers which hop fails:

- Head relay log shows nothing new -> the GCS-side gRPC client isn't even attempting the CONNECT. Points at Ray-internal GCS lease-scheduling behavior under a proxy, not this relay.
- Head relay log shows a CONNECT attempt that fails (`relay: CONNECT %s:%d dial failed: %s`) -> a real SOCKS5/tailscale reachability problem from the head to the worker's tailnet IP.
- Head relay log shows a successful CONNECT, but the worker-side raylet still never gets asked to start anything -> the problem is on the worker's inbound side -- `tailscaled` delivery or the raylet's own bind address, not the relay layer at all.

A real GCP cluster is up for this (`chia up cluster/local.yaml --yes` , `e2-highmem-8` , on-demand, `us-central1-b`) -- billed compute, not a simulation. Results land once the 6-minute patient wait resolves one way or the other.

What actually happened (2026-09-16, real cluster, real 6-minute wait)

The task sat pending the full 360s again -- same symptom, reconfirmed.

Both relay logs showed zero new activity the entire time. Not "a CONNECT that failed" -- no CONNECT attempt at all, on either the head's relay or the GCP worker's relay. This rules out both hops at once: the problem is not in `chia`'s relay/tailnet code, which was never even asked to do anything. It's upstream of both relays.

Chased this further, ruling out two more concrete candidates by reading `chia/cluster/node_setup.py` directly (not guessing):

- **Does the head's own `ray start --head` (which runs the GCS) get `grpc_proxy` exported before it starts, same as workers?** Yes -- `build_head_script()` (~line 424-455) calls the same `_grpc_proxy_exports()` helper workers get, unconditionally whenever tailnet mode is active. Ruled out.
- Two earlier candidates (SSH tunnel lifetime, node-manager-port mismatch) were already ruled out before this run -- see above.

So every `chia`-side config/wiring path checks out. The gap is narrower now than it's ever been: **something in Ray's own lease-request path (raylet-to-raylet, not GCS-mediated -- Ray's actual scheduling model has the requesting raylet dial the destination raylet directly once the GCS decides placement) isn't routing through the configured `grpc_proxy`, even though the same process's environment has it set correctly and other traffic (worker heartbeats, the opposite direction) demonstrably uses it fine.**

The most likely concrete explanation, not yet confirmed: Ray's raylet communication is implemented in its compiled C++ core, a different code path from the `grpcio` Python bindings -- `grpc_proxy` env-var handling might only be wired up for the Python-level gRPC client, not the C++ core's own channel construction. This is a real Ray-internals question, not something further `chia`-config reading will resolve. Confirming it needs either reading Ray's own C++ gRPC channel setup, or `strace/tcpdump` on a real cluster during a dispatch attempt -- both bigger, more deliberate undertakings than fits inside what's been spent today.

Cluster torn down immediately after this result (`chia down --yes`), confirmed zero instances remain (`gcloud compute instances list`).

Confirmed decisively (2026-09-16, later the same night): the strace result

Ran `strace -f -tt -e trace=network` on both raylets during a fresh forced dispatch, bracketing the exact window the task sat pending in. Head-side trace run by the user directly (`sudo timeout 105 strace ... | grep -i connect` -- `ptrace` on an already-running process needs root, and this driver's own `sudo` has no cached credential to reuse non-interactively, so that half had to be run by hand rather than automated).

Zero `connect()` syscalls on either raylet, the entire window. Not "attempted and failed" -- never attempted at all, on the head, on the worker, or on either relay. Confirmed three independent ways now (relay logs, worker strace, head strace), all agreeing.

The worker's raylet strace *does* show real, continuous activity on several already-established file descriptors during the same window -- HTTP/2 PING/PING-ACK keepalive frames and other traffic, consistent with the ordinary heartbeat/registration channel already known to work. One honest caveat worth stating plainly: HTTP/2 multiplexes multiple logical RPC streams over a single TCP connection, so a lease-assignment message riding an *already-established* connection would not show up as a new `connect()` -- raw syscall-level tracing can't distinguish "never sent" from "sent over an existing stream" without decoding the actual HTTP/2 frame contents, which needs a different, heavier tool (a real protocol dissector, e.g. Wireshark's gRPC/HTTP2 decode over a `tcpdump` capture) to settle definitively. Given the totality of the evidence -- zero new connections anywhere, the raylet's own WorkerPool never even asked to start anything -- the more likely read is a genuine gap in whether Ray's scheduler attempts placement communication at all here, not a hidden multiplexed message, but that specific residual uncertainty is real and worth stating rather than glossing over.

Cluster torn down immediately after this result.

Where this actually stands

Every `chia`-side explanation has been checked and ruled out precisely, by reading the real code, not by guessing from commit history or docs. The remaining question is squarely inside Ray's own scheduling/ placement internals under a proxied network topology -- genuinely deeper than config, and genuinely outside what reading `chia`'s repo alone can resolve further.

Two real paths from here, not yet decided: 1. **File this upstream**, either with `chia` (whose maintainers already helped resolve the earlier tailnet env-var finding this project hit) or with Ray directly. This is now an extremely well-characterized, reproducible report -- exact symptom, exact ruled-out list, exact syscall-level evidence -- the kind of report a maintainer can actually act on quickly, rather than a vague "doesn't work on GCP." 2. **Read Ray's own C++ core** (`src/ray/rpc/` in the separately-installed `ray` package, not `chia`) for how placement/lease communication is actually implemented, and whether `grpc_proxy` support was ever wired up for that specific code path. A real, deeper undertaking -- likely hours, not minutes, and may end at "this is a genuine Ray limitation" rather than something fixable from this project's side at all.

Given the hackathon deadline, (1) is very likely the better use of remaining time -- it hands the deepest part of the problem to people who actually know this code, the same pattern that already worked once this project.